TREAD: Inspectable Task-Query Reading of Frozen Multimodal Layer Representations for Advertising Response Prediction

Xin Huang
Baidu Inc.
Beijing, China
huangxin21@baidu.com

Heng Zhan
Baidu Inc.
Beijing, China
zhanheng288@gmail.com

Xiaoyang Wang
Baidu Inc.
Beijing, China
wangxiaoyang04@baidu.com

Zixuan Wei
Baidu Inc.
Beijing, China
weizixuan02@baidu.com

Yubin Chen
Baidu Inc.
Beijing, China
chenyubin02@baidu.com

Tianle Xu
Fudan University
Shanghai, China
xutianle@fudan.edu.cn

Shijie Jia
Dalian Jiaotong University
Dalian, China
jsj@djtu.edu.cn

Abstract

Video advertising predictors combine content with product and delivery features to estimate several responses. Compressing content into a single vector hides which encoder depths each task relies on and offers no direct control over the number of content tokens the predictor processes. We present TREAD (Task-query Reading across Encoder Depth), an inspectable interface in which three task queries read 21 content-layer tokens from a frozen multimodal encoder and one structured-feature token that fuses five product fields, the delivery context, the video URL, and price. The queries predict the click-through rate (CTR), the click-and-conversion rate per exposure (CTCVR), and the five-second playback rate. Training-set attention ranks the content tokens, and the resulting Top-k subsets are applied to the trained readers without retraining. Keeping 11 of the 21 tokens changes AUC by less than 0.1% on every task while reducing self-attention FLOPs by 41%; ranking controls show that the attention order identifies the final encoder layer as the essential token and, beyond this layer, yields higher CTCVR AUC than ranking-free subsets. Compared with a single-vector content model, an EARec-adapted model, and a model without content, TREAD achieves the highest AUC on all three tasks. When the five product fields are replaced by the training-vocabulary UNK embedding, its CTR and CTCVR AUC decrease by 1.3% and 2.8%, compared with 17.8% and 18.4% for the model without content. In an online A/B test on 20% of traffic, the 11-token reader yields relative gains of 10.14% in clicks, 15.90% in conversions, and 15.61% in gross merchandise volume (GMV), and increases CTR and CTCVR by 2.81% and 8.19%.

CCS Concepts

• **Information systems** → **Recommender systems**; • **Computing methodologies** → *Neural networks*.

Keywords

video advertising, multi-task prediction, frozen representations, attention readout, structured features

1 Introduction

Video advertising on the Web requires predicting several responses to the same asset: whether an exposure produces a click (CTR), a click followed by a conversion (CTCVR), or at least five seconds of playback (Play5). These targets draw on a shared video but need not use its representation in the same way. Product identity, delivery context, and price provide additional signals. A useful prediction interface must combine these heterogeneous inputs and make it possible to inspect how each target reads them.

A common interface projects a frozen encoder's final-layer vector and concatenates it with structured features. This supports efficient reuse of a large encoder, but collapses content depth before the prediction task can select among layers. Intermediate representations can instead remain separate inputs to a task readout. The design question is how to expose these states alongside structured information, and how the trained predictor allocates attention across the exposed inputs.

We propose TREAD, an interface with 21 content-layer tokens, one structured-feature token, and three learned task queries. Each selected encoder state has its own projector. The structured-feature token encodes five product-related categorical fields together with the delivery context, video URL, and price through a cross/deep network. One self-attention block mixes the sequence, and each task-query output feeds a separate response head. This arrangement preserves a named path from each encoder depth to each task readout while allowing the structured token to participate in the same attention normalization. We use the attention allocation of the trained reader to rank the content tokens and then remove lower-ranked tokens without retraining.

Task-specific reading of hidden layers has precedents in ELMo and Attention Fusion [2, 22]; IISAN also uses cached multimodal hidden states for recommendation [8], and IDProxy feeds hidden states from several layers of a frozen multimodal LLM into an advertising CTR ranker [37]. Query-conditioned attention is likewise established in advertising response models such as DIN [39]. TREAD combines these mechanisms for multi-response

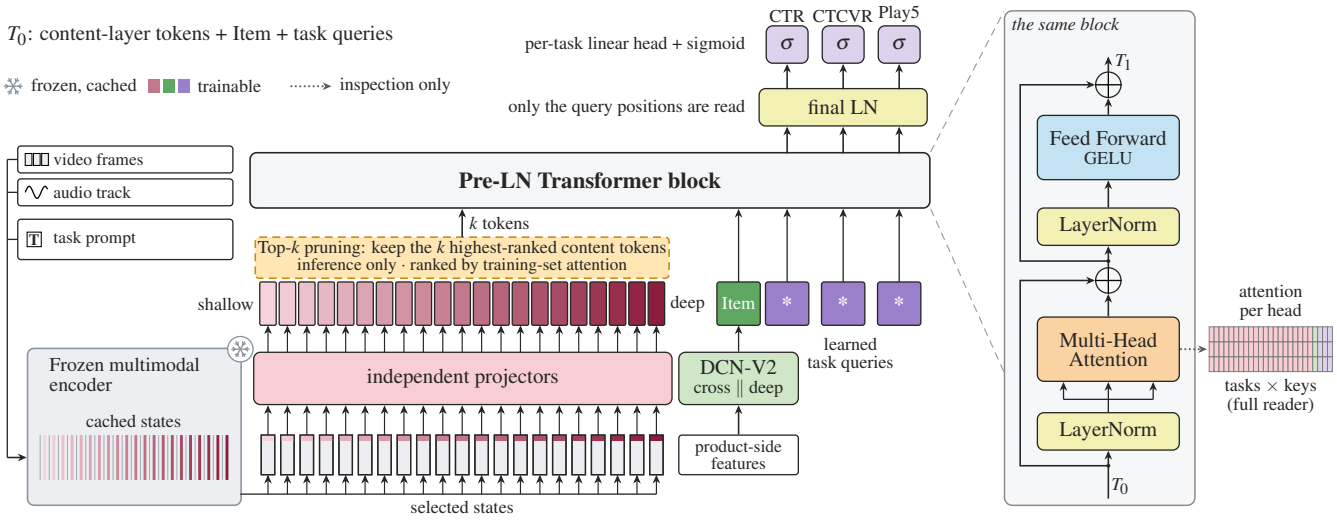


Figure 1: TREAD readout and pruning interface. Twenty-one content-layer tokens, one fused structured-feature token, and three task queries enter a single eight-head block. Training-set attention ranks the content tokens; inference-only pruning removes the lowest-ranked tokens before the block and keeps the structured-feature and query tokens. Product fields are fused into the structured-feature token before attention.

advertising prediction: each response has its own query over the content-layer tokens and the structured-feature token, and the trained reader's attention sets an inference-time token budget. The domain-adapted multimodal encoder stays frozen and cached, so TREAD operates entirely downstream of it.

Our offline evaluation has three complementary parts. We rank TREAD's 21 content tokens on the complete training set and apply four fixed Top-k subsets to the trained checkpoints without retraining. The resulting AUC is measured on the evaluation set, while self-attention FLOPs and fixed-benchmark self-attention latency quantify the cost of the readout block. Finally, a Product-UNK intervention replaces five product-related categorical fields with the UNK embedding of the training vocabulary at nominal rates from 0% to 100% and compares TREAD with a single-vector content model (Single-vector), an EARec-adapted single-vector model (EARec), and a model without content (NoContent) on the same masks. An online A/B test on 20% of traffic evaluates the 11-token reader in deployment.

Keeping the 11 highest-ranked tokens changes AUC by less than 0.1% on every task and reduces self-attention FLOPs by 41%. Ranking controls show that the attention order identifies the final layer L41 as essential, since every evaluated subset without it loses at least 8% of the full reader's CTR AUC, and that beyond L41 it yields higher CTCVR AUC than ranking-free subsets. With intact inputs, TREAD achieves the highest AUC of the four systems on all three tasks. Under full product-field replacement, its CTR and CTCVR AUC fall by 1.3% and 2.8%, the smallest drops among the four systems, whereas NoContent falls by 17.8% and 18.4%. Online, the 11-token reader yields relative gains of 2.81% in CTR, 8.19% in CTCVR, and 15.61% in GMV.

We make four contributions:

- An explicit advertising readout interface that combines separately projected content-layer tokens, one fused structured-feature token, and CTR, CTCVR, and Play5 queries.
- A training-set-guided, inference-only pruning protocol that ranks the 21 content tokens and evaluates shared Top-k subsets without retraining, together with ranking controls and reader-level cost accounting.
- A fixed-checkpoint Product-UNK analysis of four systems under shared nested masks, in which TREAD shows the smallest CTR and CTCVR drops and every content-equipped model degrades less than NoContent when product fields are missing.
- An online A/B test of the 11-token reader on 20% of traffic, with relative gains of 10.14% in clicks, 15.90% in conversions, and 15.61% in GMV.

2 Related Work

Task adaptation through hidden states. ELMo learns task-specific scalar mixtures of internal representations [22]. Attention Fusion reads intermediate and final states of a frozen encoder through task-specific queries, making layer aggregation input-dependent [2]. Layer-wise analyses of pretrained transformers further show that task-relevant information can vary substantially across depth [32], and intermediate-layer embeddings often outperform final-layer embeddings [24]. IISAN combines cached multimodal states, decoupled side adaptation, and sparse layer connections for sequential recommendation [8]. Learned latent or query tokens summarize large inputs in Perceiver [11] and read a frozen image encoder in the BLIP-2 Q-Former [15]. VQT and EVL apply learned queries to intermediate layers of frozen vision encoders [17, 27], Head2Toe selects features from all layers of a frozen backbone [6], and Dense Connector fuses multi-layer visual

features for multimodal LLMs [35]. TREAD combines hidden-layer access and query-based fusion for advertising response prediction: pooled content-layer tokens, a DCN-style structured-feature token [29], and three response queries share one block.

Advertising content representations. Early studies used multimedia and visual-appearance features for click prediction and creative ranking in display advertising [1, 4]. PEAC evaluates advertising creatives using image, slogan, OCR, and industry information [38]. HBM uses CTR supervision, including within-product listwise learning, to learn visual priors for creative ranking [30]. Category-specific visual encoders have also been used for industrial CTR prediction [18]. CG4CTR combines listwise and pointwise reward training for creative generation [33]. These works motivate using historical response signals to adapt content representations. Our frozen domain-adapted encoder is one such upstream source; this paper studies how downstream tasks read its states.

Semantic and behavioral alignment. EARec aligns modalities through generated semantic anchors and combines modality-specific adapters for recommendation [34]. Multimodal recommendation systems also combine visual, acoustic, and textual signals for micro-video personalization [31], while the content-based video CTR challenge studies large-scale video features for click prediction [28]. MOON studies multimodal product representations and behavioral post-training for search advertising [7, 36]. EM3 uses ranking and content–ID alignment [5], while the Taobao advertising study combines behavioral contrastive learning with CTR adaptation [23]. These approaches address how to obtain task-relevant content; TREAD addresses how each response reads it.

Multimodal LLM states in industrial ranking. QARM aligns multimodal LLM representations with downstream item relations and quantizes them into codes that the ranking model learns end to end [19], and LEARN maps frozen LLM item embeddings into the collaborative space [13]. Uni-CTR uses domain-specific networks that read layer-wise LLM representations for multi-domain CTR prediction [9], and a systematic study of frozen video LLMs fuses hidden states from all decoder layers for micro-video recommendation [25]. IDProxy is the closest industrial setting: it passes hidden states from several layers of a fine-tuned and then frozen multimodal LLM to a CTR ranker for cold-start items in feed and display advertising [37]. TREAD differs in giving each response its own query over the content-layer tokens and the structured-feature token within one attention block, and in using the trained reader's attention to set an inference-time token budget.

Multi-task response prediction. ESMM estimates CTR and CTCVR over the entire exposure space [21], and MMoE and PLE share experts across tasks through task-specific gates [20, 26]. TREAD shares one token sequence across responses and gives each response its own attention query.

3 An Inspectable Token Readout

3.1 Prediction unit and inputs

Let v be a video, p its structured product fields, and s the delivery context. The predictor returns $\hat{r}_t(v, p, s)$ for each of three tasks. CTR counts clicks per exposure, CTCVR counts click-and-conversion events per exposure, and Play5 counts playback of at least five seconds per exposure. CTCVR thus uses exposures rather than clicks as its denominator, as in entire-space multi-task modeling [21].

The records aggregate exposures by (s, URL) and retain representative product fields. The prediction unit is therefore a context–video row, with one prediction shared by its exposures. The model receives no empirical response rate of the target video as input.

3.2 Content-layer tokens

A frozen multimodal encoder supplies cached states $h_0, \dots, h_{41} \in \mathbb{R}^{2560}$. Each state is a masked mean over valid video, audio, and prompt positions. Slots 0–40 use decoder block outputs before final normalization; slot 41 uses the output of the last block after final normalization. The task-query readout selects 21 states, taking every other encoder state:

$$I_{21} = [41, 39, 37, \dots, 5, 3, 1]. \tag{1}$$

We refer to these as $L41, L39, \dots, L1$, where L41 is the final layer; figures list them from shallow (L1) to deep (L41). Each state is projected independently into a common 256-dimensional space:

$$z_k = \mathrm{LN}_2\Big(W_{2,k}\,\mathrm{GELU}(W_{1,k}\,\mathrm{LN}_1(h_{i_k}))\Big), \tag{2}$$

where the dimensions are $2560 \to 512 \to 256$. As is common for multimodal item features [5, 37], we call z_k a content-layer token (content token for short); it summarizes the video and audio at one encoder depth. Independent projectors accommodate depth-specific feature distributions and keep each selected state separately addressable. The cached states receive no gradients during downstream training, whereas the projectors are trained.

3.3 One structured-feature token

The structured input contains five product-related categorical fields (shop, standard product unit, brand, product, and company IDs), plus the transaction type, video URL, log-price, and a price-missing indicator. This combination of categorical and continuous features follows standard feature-interaction models for CTR prediction [10, 29]. The transaction-type embedding encodes the delivery context. All seven categorical embeddings have width 32, yielding $7 \times 32 + 2 = 226$ input dimensions. Vocabularies are built from the downstream training data. Index zero holds a randomly initialized, trainable UNK embedding; it is neither a padding index nor fixed to zero.

Let x be the concatenated structured input. Three cross layers and a deep branch encode it:

$$x^{(0)} = x, \qquad x^{(l+1)} = x \odot (W_l x^{(l)} + b_l) + x^{(l)}. \tag{3}$$

The deep branch has widths $226 \to 512 \to 256 \to 128$ with ReLU activations. The cross output and deep output are concatenated and projected to a 256-dimensional layer-normalized structured-feature token e_{item}, labeled Item in the figures. Thus, the product fields interact inside the structured encoder before entering attention. In the attention analyses, the Item key represents this entire group, including the delivery context.

3.4 Task queries and the readout

The input sequence is

$$T_0 = [z_{L41}, z_{L39}, \dots, z_{L1}, e_{\text{item}}, q_{\text{CTR}}, q_{\text{CTCVR}}, q_{\text{Play5}}]. \quad (4)$$

All 25 tokens have width 256 in the full reader. The reader is a single pre-layer-normalization (pre-LN) self-attention block with eight heads of width 32. With MHA denoting multi-head attention, its residual updates are

$$U = T_0 + \text{Dropout}(\text{MHA}(\text{LN}(T_0))), \quad (5)$$

$$T_1 = U + \text{Dropout}(\text{FFN}(\text{LN}(U))). \quad (6)$$

The feed-forward network has hidden width 512 and GELU activation; dropout is 0.1 during training and disabled at inference. A final layer normalization precedes independent linear logit heads on the three output query positions. A sigmoid yields each task probability.

For the full reader, the attention weight of head h from task query t to key j is

$$a_{tj}^{(h)} = \frac{\exp \ell_{tj}^{(h)}}{\sum_{u=1}^{25} \exp \ell_{tu}^{(h)}}, \qquad \ell_{tj}^{(h)} = \frac{\langle Q_h \bar{q}_t, K_h \bar{T}_{0,j} \rangle}{\sqrt{32}}, \quad (7)$$

where bars denote token-wise layer normalization and the projections include their learned biases. In a Top-k intervention, the unselected content tokens are physically removed before attention, so the softmax runs over $k + 4$ keys; the structured-feature and query tokens are always retained. The Product-UNK intervention retains all 25 keys and measures the prediction sensitivity of the fixed model.

This design makes token-level reading directly measurable. Because attention weights alone are not faithful explanations [12], we treat them as allocation measurements and test every attention-derived ranking by intervention, removing tokens and measuring the change in AUC (Section 5).

3.5 Training objective

Training is performed at the exposure level. Each exposure forms one binary example per task, labeled positive if it leads to a click (CTR), a click followed by a conversion (CTCVR), or at least five seconds of playback (Play5). For record i, let n_i be its exposure count and c_{it} the number of its exposures that are positive for task t. All exposures of a record share the prediction $\hat{r}_{it}$, so the exposure-level binary cross-entropy (BCE) gives the loss

$$\mathcal{L} = \sum_t \lambda_t \frac{\sum_i \left[c_{it} \, \text{BCE}(\hat{r}_{it}, 1) + (n_i - c_{it}) \, \text{BCE}(\hat{r}_{it}, 0) \right]}{\sum_i n_i}, \quad (8)$$

with task weights $(\lambda_{\text{CTR}}, \lambda_{\text{CTCVR}}, \lambda_{\text{Play5}}) = (1, 2, 1)$. Evaluation uses the same exposure-level units (Section 4.1).

4 Evaluation Protocol

Both interventions evaluate fixed checkpoints without retraining (Figure 2). Top-k pruning fixes one content-token ranking on the training set before evaluation, while Product-UNK replacement changes only product-field entries on the evaluation set.

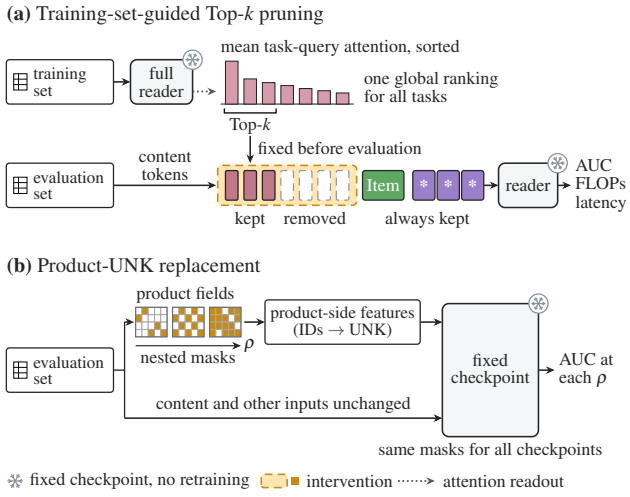


Figure 2: Fixed-checkpoint interventions. (a) Training-set-guided Top-k pruning: task-query attention of the full reader on the training set yields one global content-token ranking, fixed before evaluation. On the evaluation set, lower-ranked content tokens are removed before the block, while the structured-feature token and task queries are always kept. (b) Product-UNK replacement: nested masks replace product-field entries with the UNK embedding at rate ρ; content and all other inputs stay as recorded, and every compared checkpoint receives the same masks. Token and cell counts are schematic.

4.1 Evaluation set and outcomes

The data come from two anonymized delivery contexts of an industrial video-advertising platform. Downstream training and evaluation use successive historical windows. The evaluation set retains URLs whose total exposures over the evaluation window and both contexts reach a fixed threshold; all context-specific rows of a retained URL are kept, even when one context alone falls below the threshold. One fixed evaluation set serves all reported comparisons.

Checkpoint selection and early stopping use this evaluation set. All interventions are then applied to the selected checkpoints: rankings, Top-k subsets, and replacement rates are fixed before evaluation and are never used to reselect a checkpoint.

All three tasks are evaluated with exposure-level binary ROC-AUC on the complete evaluation set. An aggregated row with prediction $\hat{r}_{it}$ supplies c_{it} positive and $n_i - c_{it}$ negative exposures at that prediction. Pair counting with half credit for ties is therefore equivalent to expanding each row into binary observations. We report one pooled exposure-level ROC-AUC (pooled AUC) for each task. In the text, AUC changes are expressed relative to a reference AUC, namely that of the full reader or a model's own 0% value. SD denotes the sample standard deviation across independent training runs.

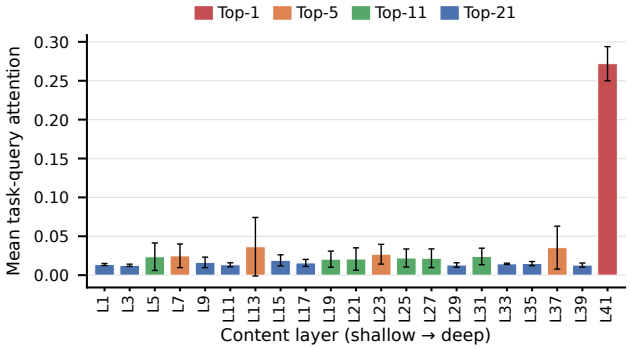


Figure 3: TREAD content-token ranking on the complete training set. Bars show the mean post-softmax task-query attention weight over rows, heads, and the three tasks; error bars show sample SD over independent training runs. Colors mark the nested Top-1, Top-5, Top-11, and Top-21 sets used for inference-only pruning in Table 1.

4.2 Product-field replacement

The Product-UNK analysis evaluates fixed checkpoints on the evaluation set rather than the training set. It measures prediction sensitivity to missing product information while preserving the observed records and all non-product inputs.

For each of five mask realizations, each row i, and each product field f in Section 3.3, we draw an independent uniform variable u_{if}. At nominal rate $\rho \in \{0, 0.25, 0.5, 0.75, 1\}$, the intervention is

$$x'_{if} = \begin{cases} 0, & u_{if} < \rho, \\ x_{if}, & \text{otherwise.} \end{cases} \tag{9}$$

The same nested draws are reused across rates and all compared checkpoints. Originally unknown entries remain unknown; selected entries are mapped to the UNK index zero of the training vocabulary and hence to its trainable embedding. Transaction type, URL, price and its missing indicator, labels, cached content, and model parameters stay unchanged.

Because every rate and checkpoint uses the same draws, AUC comparisons are paired at the row level. Results aggregate the five mask realizations and three training runs per system. The main text pools the two delivery contexts; Appendix A reports them separately and decomposes the variation into model-run and mask components.

4.3 Compared systems

The Product-UNK analysis compares four downstream systems that share the structured encoder inputs of Section 3.3, the vocabularies built from training data, and the three response heads. *TREAD* is the full reader with 21 content-layer tokens. *Single-vector* is the common interface described in the introduction: a 2560→512→256 projector maps the final-layer state of the same frozen encoder to one content vector, which is concatenated with the 226-dimensional structured input before the cross and deep branches, giving 482 input dimensions. *EARec* applies the same fusion to the final-layer state of an encoder adapted with the EARec alignment method [34]. *NoContent* uses the structured input alone. Each reported model result is the mean over three independent downstream training runs.

Table 1: Inference-only pruning of the fixed TREAD checkpoints on the evaluation set. AUC is mean ± sample SD across three training runs. FLOPs (relative to Top-21) and latency cover the reader self-attention block; latency is the mean batch time of the three trained readers. Top-11* is the configuration deployed online.

Kept	CTR AUC	CTCVR AUC	Play5 AUC	FLOPs	ms
Top-1	0.6609 ± 0.0025	0.6909 ± 0.0088	0.6444 ± 0.0024	0.193	1.12
Top-5	0.6662 ± 0.0006	0.7077 ± 0.0080	0.6494 ± 0.0015	0.349	1.18
Top-11*	0.6670 ± 0.0003	0.7171 ± 0.0017	0.6500 ± 0.0017	0.589	1.59
Top-21	0.6672 ± 0.0005	0.7174 ± 0.0021	0.6505 ± 0.0019	1.000	1.96

5 Training-Set-Guided Token Pruning

The full TREAD reader attends over 21 content-layer tokens, one structured-feature token, and three task queries. We compute the post-softmax attention weight assigned to each content token on every training row, head, and task query with unmodified inputs (0% Product-UNK). For training run m, the score of content token j is

$$s_m(j) = \frac{1}{3N_m H} \sum_{i=1}^{N_m} \sum_{h=1}^{H} \sum_{t \in \{\text{CTR,CTCVR,Play5}\}} a_{i,tj}^{(h)}, \tag{10}$$

where $a_{i,tj}^{(h)}$ is the attention weight of Eq. (7) on training row i and $H = 8$. The scores from the independent training runs are averaged per token, and the 21 content tokens are ranked by this mean. Ties are broken in shallow-to-deep order. The structured-feature and query keys are excluded from the ranking, and no exposure weighting or delivery-context split is used. This fixed, post-training intervention is related to learned token-pruning methods that remove low-utility tokens to reduce transformer inference cost [14] and to attention-guided token reorganization in vision transformers [16], whereas TREAD ranks its content-layer tokens once after training and evaluates a shared subset without retraining.

Figure 3 shows the resulting layer scores with sample SD across independent training runs. Figure 4 shows how each task query departs from the mean allocation of the three queries on the evaluation rows: the Play5 query attends more to L41 and the structured-feature token, the CTCVR query attends less to L41 and more to its own query key, and the CTR query attends less to the structured-feature token. We use one global ranking for all three tasks and retain the top $k \in \{1, 5, 11, 21\}$ content tokens. For each fixed TREAD checkpoint, the unselected projected tokens are physically removed before the self-attention block; the structured-feature token and all three query tokens remain. The retained tokens keep their layer identity and order. Pruning involves no checkpoint reselection, no tuning of k on the evaluation set, and no retraining.

Table 1 reports the accuracy–cost trade-off, and Figure 6 in Appendix B plots it. Keeping the 11 highest-ranked tokens changes AUC by less than 0.1% on every task while reducing self-attention FLOPs by 41% (to 0.589 of the full reader) and latency by 19% (from

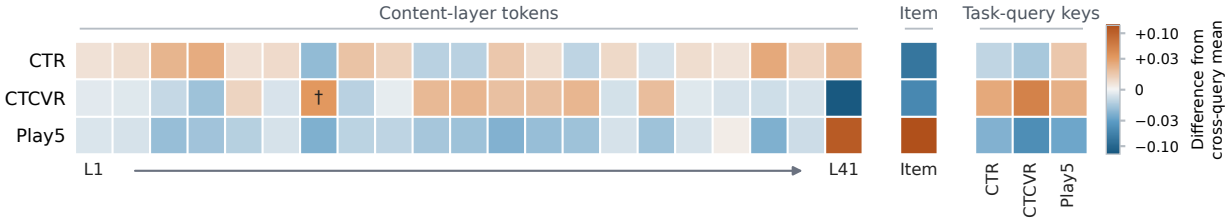


Figure 4: Task-specific attention of the full TREAD reader on the evaluation rows with unmodified inputs. Each cell is one task query's mean attention weight minus the mean over the three task queries, for the 21 content-layer tokens, the structured-feature token (Item), and the three task-query keys; means are taken over rows, heads, and three independent training runs. Orange marks more attention than the cross-query mean and blue marks less, on a square-root color scale. Content-layer tokens run from the first selected layer (L1) to the last (L41). Over content tokens, the cross-query mean agrees with the training-set ranking in Figure 3 (Spearman $\rho = 0.999$). †: a difference of at least 0.03 whose sign is not shared by all three runs.

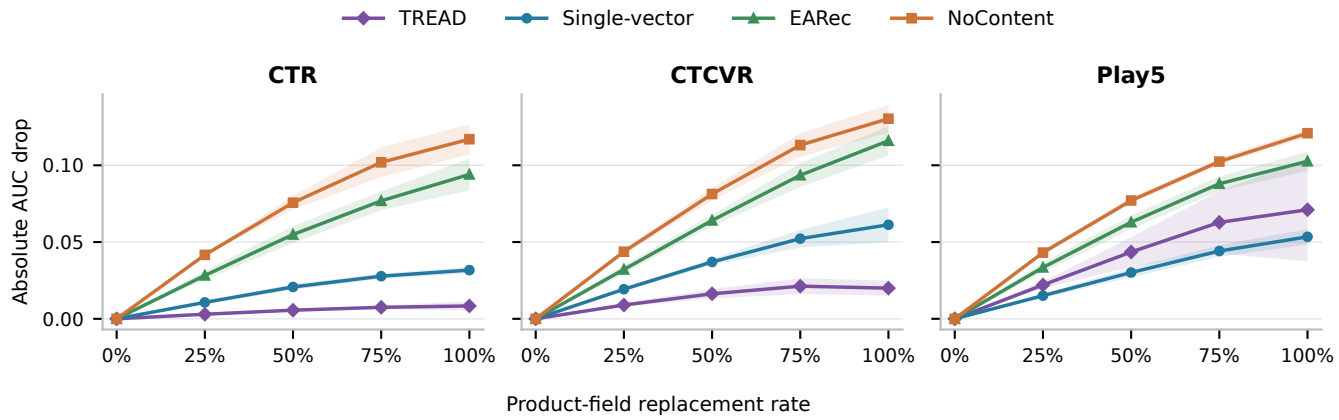


Figure 5: Product-UNK sensitivity of TREAD, Single-vector, EARec, and NoContent under the same nested masks. Each curve is a model's absolute pooled-AUC drop from its own 0% value, averaged over three training runs and five mask realizations; bands show the model-run SD. Table 3 lists the absolute AUCs.

1.96 to 1.59 ms); keeping L41 alone lowers CTR AUC by 0.9% and CTCVR AUC by 3.7%. FLOPs count the projections and query–key/value products in the self-attention block of the reader; they exclude the frozen encoder, content projectors, structured encoder, and FFN. Latency is measured with a fixed-batch benchmark on the same hardware and with the same timing protocol for all k, and covers only the self-attention forward pass. We report the mean self-attention latency over the three trained readers. Because removed tokens never enter the block, a Top-k reader also needs only k cached states and k content projectors, a further saving not included in these figures.

Ranking controls. Table 2 compares the attention ranking with ranking-free subsets of the same size on the same fixed checkpoints and evaluation set. The ranking's first choice, L41, is essential: every evaluated subset without L41 loses at least 8% of the full reader's CTR AUC, and the best single token other than L41 reaches 0.6027 CTR AUC, compared with 0.6609 for L41 alone. Once L41 is retained, CTR and Play5 are largely insensitive to the remaining tokens: L41 combined with random, deepest, or evenly spaced tokens stays within about 0.1% of the attention Top-k AUC.

Table 2: Ranking controls for inference-only pruning of the fixed TREAD checkpoints. Each rule keeps k content-layer tokens; the structured-feature and query tokens are always kept. Values are pooled AUC averaged over three training runs; † rows also average 20 random subsets drawn once and shared by all runs. Per-task attention reads each task head from a pass that keeps that task's own training-set Top-k. The full reader reaches 0.6672 / 0.7174 / 0.6505.

	$k = 5$			$k = 11$		
Subset rule	CTR	CTCVR	Play5	CTR	CTCVR	Play5
Attention Top-k	0.6662	0.7077	0.6494	0.6670	0.7171	0.6500
Per-task attention	0.6658	0.7128	0.6496	0.6669	0.7172	0.6503
L41 + random†	0.6655	0.7043	0.6489	0.6667	0.7119	0.6500
Deepest k	0.6661	0.7039	0.6493	0.6669	0.7123	0.6499
Evenly spaced	0.6656	0.7038	0.6489	0.6671	0.7135	0.6498
Random†	0.6267	0.6750	0.6365	0.6354	0.6868	0.6392
Lowest-ranked	0.6009	0.6499	0.6278	0.6061	0.6564	0.6298

For CTCVR, the attention order is beneficial: it exceeds these three

rules by 0.5–0.7% in CTCVR AUC; at $k = 11$, none of the 20 random L41 subsets reaches it, and it exceeds their mean in each of the three training runs. This is consistent with Figure 4, where the CTCVR query attends less to L41 than the other queries, and with Table 1, where CTCVR loses most when only L41 is kept. A ranking computed from the CTCVR query alone raises Top-5 CTCVR AUC by 0.7%, from 0.7077 to 0.7128, closing about half of the remaining gap to the full reader; per-task rankings change CTR and Play5 by less than 0.1% and CTCVR at $k = 11$ by about 0.01%. This gain comes at the cost of one reader pass per task, which is not included in Table 1. When the ranking is computed separately for each training run, all three runs place L41 first.

6 Prediction under Product-UNK Replacement

6.1 Sensitivity to missing product features

This analysis asks whether content-equipped systems retain more of their ranking quality than NoContent when the five product fields become unavailable at inference. Table 3 and Figure 5 report TREAD, Single-vector, EARec, and NoContent at the same five nested Product-UNK rates. Each value is pooled AUC on the evaluation set, averaged over independent training runs and mask realizations. With intact inputs, TREAD achieves the highest AUC of the four systems on all three tasks (0% row of Table 3). Because the systems start from different 0% values, we compare each model's relative drop from its own 0% AUC.

All three content-equipped systems degrade less than NoContent on every task. At 100% replacement, NoContent loses 17.8%, 18.4%, and 19.0% of its CTR, CTCVR, and Play5 AUC. TREAD loses 1.3% and 2.8% on CTR and CTCVR, the smallest drops among the four systems, and retains 0.659 and 0.697 AUC, whereas NoContent falls to 0.541 and 0.580; even without any product field, its CTR AUC remains on par with that of NoContent with all fields intact (0.658). The Play5 query attends more heavily to the structured-feature token: with unmodified inputs, it allocates 0.337 of its attention to this token, compared with 0.134 for CTR and 0.159 for CTCVR (Figure 4, Appendix A). Consistent with this allocation, TREAD's Play5 drop (10.9%) exceeds its CTR and CTCVR drops and is the second smallest among the four systems, after that of Single-vector (8.2%). EARec loses 14.2–16.3% across tasks, closer to NoContent than to the other two content-equipped systems. Every content-equipped system also drops less than NoContent at each intermediate rate (Figure 5), and Appendix A reports results by delivery context and URL slice, together with the sources of variation.

Because records, cached content, and parameters stay fixed, these drops measure how strongly each trained system relies on product fields at inference.

7 Online A/B Test

We deployed TREAD with its 11 highest-ranked content tokens (Top-11 in Table 1) in an online A/B test that assigned 20% of traffic to the treatment group. Table 4 lists the relative changes of the treatment group with respect to the control group. Clicks rose by 10.14%, conversions by 15.90%, and GMV by 15.61%. Exposures rose by 7.13%, and the per-exposure rates also increased: CTR by 2.81% and CTCVR by 8.19%. The gains therefore stem from both additional exposures and higher response rates per exposure, and they are largest for conversion-related outcomes.

8 Discussion

Upstream encoder. The content cache is produced by a multimodal encoder that takes video, audio, and a task prompt as input, followed by masked pooling in evaluation mode. The encoder is adapted with response supervision through parameter-efficient updates, while its modality components remain frozen. The adaptation uses a listwise objective [3] with normalized response targets and softmax-normalized scores. Downstream, the cached states serve as fixed inputs.

Extensions. The token interface extends naturally. Separate product-field and user-side tokens would add further named paths, and additional interaction blocks would allow structured information to modify content or query states before the task queries read them.

9 Conclusion

We presented TREAD, a task-query reader that exposes 21 content-layer tokens from a frozen multimodal encoder and one structured-feature token to three response queries in a single attention block. Its training-set attention sets a token budget without retraining: keeping 11 of the 21 tokens changes AUC by less than 0.1% while reducing self-attention FLOPs by 41%, and the ranking identifies the final encoder layer as essential and, beyond this layer, yields higher CTCVR AUC than ranking-free subsets. Among the four compared systems, TREAD achieves the highest AUC on all three tasks and, under full product-field replacement, loses the least CTR and CTCVR AUC. In an online A/B test on 20% of traffic, the 11-token reader raised clicks by 10.14%, conversions by 15.90%, and GMV by 15.61%. Exposing encoder depth as named tokens thus makes the use of content in advertising response prediction both inspectable and prunable.

References

[1] Javad Azimi, Ruofei Zhang, Yang Zhou, Vidhya Navalpakkam, Jianchang Mao, and Xiaoli Z. Fern. 2012. The Impact of Visual Appearance on User Response in Online Display Advertising. In *Proceedings of the 21st International World Wide Web Conference*. 457–458. doi:10.1145/2187980.2188075

[2] Jin Cao, Chandana Satya Prakash, and Wael Hamza. 2022. Attention Fusion: a light yet efficient late fusion mechanism for task adaptation in NLU. In *Findings of the Association for Computational Linguistics: NAACL 2022*. 857–866. doi:10.18653/v1/2022.findings-naacl.64

[3] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. 2007. Learning to Rank: From Pairwise Approach to Listwise Approach. In *Proceedings of the 24th International Conference on Machine Learning*. doi:10.1145/1273496.1273513

[4] Haibin Cheng, Roelof van Zwol, Javad Azimi, Eren Manavoglu, Ruofei Zhang, Yang Zhou, and Vidhya Navalpakkam. 2012. Multimedia Features for Click Prediction of New Ads in Display Advertising. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 777–785. doi:10.1145/2339530.2339652

[5] Xiuqi Deng, Lu Xu, Xiyao Li, Jinkai Yu, Erpeng Xue, Zhongyuan Wang, Di Zhang, Zhaojie Liu, Guorui Zhou, Yang Song, Na Mou, Shen Jiang, and Han Li. 2024. End-to-end training of Multimodal Model and ranking Model. *arXiv preprint arXiv:2404.06078* (2024). https://arxiv.org/abs/2404.06078

[6] Utku Evci, Vincent Dumoulin, Hugo Larochelle, and Michael C. Mozer. 2022. Head2Toe: Utilizing Intermediate Representations for Better Transfer Learning. In *Proceedings of the 39th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 162)*. 6009–6033. https://proceedings.mlr.press/v162/evci22a.html

Table 3: Pooled AUC under nested Product-UNK masks on the evaluation set, mean over independent training runs and mask realizations. The last row is the absolute AUC drop from each model's own 0% value to 100%. Model-run and mask-realization SDs are reported separately in Appendix A.

Replacement rate	TREAD			Single-vector			EARec			NoContent		
	CTR	CTCVR	Play5	CTR	CTCVR	Play5	CTR	CTCVR	Play5	CTR	CTCVR	Play5
0%	0.6672	0.7174	0.6505	0.6640	0.7149	0.6494	0.6615	0.7115	0.6430	0.6584	0.7104	0.6350
25%	0.6642	0.7084	0.6285	0.6533	0.6957	0.6344	0.6333	0.6793	0.6096	0.6168	0.6667	0.5919
50%	0.6616	0.7011	0.6070	0.6432	0.6779	0.6193	0.6066	0.6474	0.5801	0.5827	0.6291	0.5579
75%	0.6597	0.6961	0.5877	0.6362	0.6627	0.6053	0.5847	0.6179	0.5550	0.5565	0.5973	0.5326
100%	0.6588	0.6974	0.5794	0.6322	0.6537	0.5960	0.5675	0.5955	0.5404	0.5414	0.5801	0.5141
Drop, 0%→100%	0.0084	0.0200	0.0711	0.0317	0.0613	0.0534	0.0940	0.1160	0.1026	0.1170	0.1304	0.1209

Table 4: Online A/B test of TREAD with the Top-11 content tokens on 20% of traffic: relative change of the treatment group with respect to the control group.

Click	Conversion	GMV
+10.14%	+15.90%	+15.61%
Exposure	**CTR**	**CTCVR**
+7.13%	+2.81%	+8.19%

[7] Chenghan Fu, Daoze Zhang, Yukang Lin, Zhanheng Nie, Xiang Zhang, Jianyu Liu, Yueran Liu, Wanxian Guan, Pengjie Wang, Jian Xu, and Bo Zheng. 2025. MOON Embedding: Multimodal Representation Learning for E-commerce Search Advertising. *arXiv preprint arXiv:2511.11305* (2025). https://arxiv.org/abs/2511.11305

[8] Junchen Fu, Xuri Ge, Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, Jie Wang, and Joemon M. Jose. 2024. IISAN: Efficiently Adapting Multimodal Representation for Sequential Recommendation with Decoupled PEFT. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. doi:10.1145/3626772.3657725

[9] Zichuan Fu, Xiangyang Li, Chuhan Wu, Yichao Wang, Kuicai Dong, Xiangyu Zhao, Mengchen Zhao, Huifeng Guo, and Ruiming Tang. 2025. A Unified Framework for Multi-Domain CTR Prediction via Large Language Models. *ACM Transactions on Information Systems* 43, 5 (2025), 1–33. doi:10.1145/3698878

[10] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*. 1725–1731. doi:10.24963/ijcai.2017/239

[11] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. 2021. Perceiver: General Perception with Iterative Attention. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*. 4651–4664. https://proceedings.mlr.press/v139/jaegle21a.html

[12] Sarthak Jain and Byron C. Wallace. 2019. Attention is not Explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 3543–3556. doi:10.18653/v1/N19-1357

[13] Jian Jia, Yipei Wang, Yan Li, Honggang Chen, Xuehan Bai, Zhaocheng Liu, Jian Liang, Quan Chen, Han Li, Peng Jiang, and Kun Gai. 2025. LEARN: Knowledge Adaptation from Large Language Model to Recommendation for Practical Industrial Application. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 11861–11869. doi:10.1609/aaai.v39i11.33291

[14] Sehoon Kim, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun, and Kurt Keutzer. 2022. Learned Token Pruning for Transformers. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 784–794. doi:10.1145/3534678.3539260

[15] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. 19730–19742. https://proceedings.mlr.press/v202/li23q.html

[16] Youwei Liang, Chongjian Ge, Zhan Tong, Yibing Song, Jue Wang, and Pengtao Xie. 2022. Not All Patches are What You Need: Expediting Vision Transformers via Token Reorganizations. In *International Conference on Learning Representations*. https://arxiv.org/abs/2202.07800

[17] Ziyi Lin, Shijie Geng, Renrui Zhang, Peng Gao, Gerard de Melo, Xiaogang Wang, Jifeng Dai, Yu Qiao, and Hongsheng Li. 2022. Frozen CLIP Models are Efficient Video Learners. In *Computer Vision – ECCV 2022 (Lecture Notes in Computer Science)*. 388–404. doi:10.1007/978-3-031-19833-5_23

[18] Hu Liu, Jing Lu, Hao Yang, Xiwei Zhao, Sulong Xu, Hao Peng, Zehua Zhang, Wenjie Niu, Xiaokun Zhu, Yongjun Bao, and Weipeng Yan. 2020. Category-Specific CNN for Visual-Aware CTR Prediction at JD.com. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2686–2696. doi:10.1145/3394486.3403319

[19] Xinchen Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, Changqing Qiu, Jiaqi Zhang, Xu Zhang, Zhiheng Yan, Jingming Zhang, Simin Zhang, Mingxing Wen, Zhaojie Liu, and Guorui Zhou. 2025. QARM: Quantitative Alignment Multi-Modal Recommendation at Kuaishou. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5915–5922. doi:10.1145/3746252.3761502

[20] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi. 2018. Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1930–1939. doi:10.1145/3219819.3220007

[21] Xiao Ma, Liqin Zhao, Guan Huang, Zhi Wang, Zelin Hu, Xiaoqiang Zhu, and Kun Gai. 2018. Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate. In *Proceedings of the 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1137–1140. doi:10.1145/3209978.3210104

[22] Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep Contextualized Word Representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 2227–2237. doi:10.18653/v1/N18-1202

[23] Xiang-Rong Sheng, Feifan Yang, Litong Gong, Biao Wang, Zhangming Chan, Yujing Zhang, Yueyao Cheng, Yong-Nan Zhu, Tiezheng Ge, Han Zhu, Yuning Jiang, Jian Xu, and Bo Zheng. 2024. Enhancing Taobao Display Advertising with Multimodal Representations: Challenges, Approaches and Insights. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4858–4865. doi:10.1145/3627673.3680068

[24] Oscar Skean, Md Rifat Arefin, Dan Zhao, Niket Nikul Patel, Jalal Naghiyev, Yann LeCun, and Ravid Shwartz-Ziv. 2025. Layer by Layer: Uncovering Hidden Representations in Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. 55854–55875. https://proceedings.mlr.press/v267/skean25a.html

[25] Huatuan Sun, Yunshan Ma, Changguang Wu, Yanxin Zhang, Pengfei Wang, and Xiaoyu Du. 2026. Frozen LVLMs for Micro-Video Recommendation: A Systematic Study of Feature Extraction and Fusion. In *Proceedings of the 2026 International Conference on Multimedia Retrieval*. 40–49. doi:10.1145/3805622.3810736

[26] Hongyan Tang, Junning Liu, Ming Zhao, and Xudong Gong. 2020. Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations. In *Fourteenth ACM Conference on Recommender Systems*. 269–278. doi:10.1145/3383313.3412236

[27] Cheng-Hao Tu, Zheda Mai, and Wei-Lun Chao. 2023. Visual Query Tuning: Towards Effective Usage of Intermediate Representations for Parameter and Memory Efficient Transfer Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7725–7735. https://openaccess.thecvf.com/content/CVPR2023/html/Tu_Visual_Query_Tuning_Towards_Effective_Usage_of_Intermediate_Representations_for_CVPR_2023_paper.html

[28] Peng Wang, Yunsheng Jiang, Chunxu Xu, and Xiaohui Xie. 2019. Overview of Content-Based Click-Through Rate Prediction Challenge for Video Recommendation. In *Proceedings of the 27th ACM International Conference on Multimedia*. 2593–2596. doi:10.1145/3343031.3356085

[29] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Lichan Hong, and Ed Chi. 2021. DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems. In *Proceedings of the Web Conference 2021*. 1785–1797. doi:10.1145/3442381.3450078

[30] Shiyao Wang, Qi Liu, Tiezheng Ge, Defu Lian, and Zhiqiang Zhang. 2021. A Hybrid Bandit Model with Visual Priors for Creative Ranking in Display Advertising. In *Proceedings of the Web Conference 2021*. 2324–2334. doi:10.1145/3442381.3449910

[31] Yinwei Wei, Xiang Wang, Liqiang Nie, Xiangnan He, Richang Hong, and Tat-Seng Chua. 2019. MMGCN: Multi-modal Graph Convolution Network for Personalized Recommendation of Micro-video. In *Proceedings of the 27th ACM International Conference on Multimedia*. 1437–1445. doi:10.1145/3343031.3351034

[32] Shuo Xie, Jiahao Qiu, Ankita Pasad, Li Du, Qing Qu, and Hongyuan Mei. 2022. Hidden State Variability of Pretrained Language Models Can Guide Computation Reduction for Transfer Learning. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 5750–5768. doi:10.18653/v1/2022.findings-emnlp.422

[33] Hao Yang, Jianxin Yuan, Shuai Yang, Linhe Xu, Shuo Yuan, and Yifan Zeng. 2024. A New Creative Generation Pipeline for Click-Through Rate with Stable Diffusion Model. In *Companion Proceedings of the ACM Web Conference 2024*. 180–189. doi:10.1145/3589335.3648315

[34] Shenghao Yang, Weizhi Ma, Zhiqiang Guo, Min Zhang, Haiyang Wu, Junjie Zhai, Chunhui Zhang, and Yuekui Yang. 2025. Explainable Multi-Modality Alignment for Transferable Recommendation. In *Proceedings of the ACM Web Conference 2025*. 2076–2084. doi:10.1145/3696410.3714733

[35] Huanjin Yao, Wenhao Wu, Taojiannan Yang, Yuxin Song, Mengxi Zhang, Haocheng Feng, Yifan Sun, Zhiheng Li, Wanli Ouyang, and Jingdong Wang. 2024. Dense Connector for MLLMs. In *Advances in Neural Information Processing Systems 37*. 33108–33140. doi:10.52202/079017-1043

[36] Daoze Zhang, Chenghan Fu, Zhanheng Nie, Jianyu Liu, Wanxian Guan, Yuan Gao, Jun Song, Pengjie Wang, Jian Xu, and Bo Zheng. 2026. MOON: Generative MLLM-based Multimodal Representation Learning for E-commerce Product Understanding. In *Proceedings of the International Conference on Web Search and Data Mining*. https://arxiv.org/abs/2508.11999 Public author version, arXiv:2508.11999v5.

[37] Yubin Zhang, Haiming Xu, Guillaume Salha-Galvan, Ruiyan Han, Feiyang Xiao, Yanhua Huang, Li Lin, Yang Luo, and Yao Hu. 2026. IDProxy: CTR Prediction with Multimodal LLMs for Cold-Start Recommendation at Xiaohongshu. In *Proceedings of the 20th ACM Conference on Recommender Systems*. 1193–1198. doi:10.1145/3773078.3831874

[38] Zhichen Zhao, Lei Li, Bowen Zhang, Meng Wang, Yuning Jiang, Li Xu, Fengkun Wang, and Weiying Ma. 2019. What You Look Matters?: Offline Evaluation of Advertising Creatives for Cold-start Problem. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 2605–2613. doi:10.1145/3357384.3357813

[39] Guorui Zhou, Chengru Zhu, Xiaoqiang Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1059–1068. https://www.kdd.org/kdd2018/accepted-papers/view/deep-interest-network-for-click-through-rate-prediction

A Product-UNK Diagnostics

This appendix complements Section 6 with additional diagnostics. All values come from the same fixed checkpoints and nested masks as Table 3.

Protocol details. Each of the four systems is evaluated with three independent downstream training runs and five mask realizations, giving 15 model–mask combinations per system, each evaluated at five replacement rates. Every mask realization is generated once and shared across all systems and rates. At 0%, each replay reproduces the AUC recorded for its checkpoint at selection time to within 10^{-5} in both delivery contexts, confirming that the evaluation inputs, vocabularies, content caches, and weights match those used for selection. TREAD and Single-vector read the same cached frozen-encoder states, NoContent reads no content, and EARec reads its own all-layer cache and uses only its final-layer slot. For each checkpoint, we record its configuration, data and vocabulary versions, cache version, and selected epoch.

URL slice and delivery context. Table 5 breaks down the drop from 0% to 100% by delivery context and by whether the URL appears in the training vocabulary. TREAD and Single-vector drop less than NoContent in every cell, including URLs absent from training. EARec drops less in every all-URL cell, but on unseen URLs its context-B CTCVR drop exceeds that of NoContent.

Table 5: Absolute Product-UNK AUC drop from each model's own 0% value to 100%, by URL slice and delivery context. Rows labeled All cover the whole evaluation set; rows labeled Unseen keep only URLs absent from the training vocabulary. A and B are the two anonymized delivery contexts; each cell is exposure-level ROC-AUC within one context. Values are means over three training runs and five mask realizations.

Model	URL slice	CTR		CTCVR		Play5	
		A	B	A	B	A	B
TREAD	All	0.0053	0.0094	0.0128	0.0225	0.0299	0.0724
	Unseen	0.0083	0.0064	0.0158	0.0162	0.0118	0.0314
Single-vector	All	0.0240	0.0357	0.0385	0.0702	0.0289	0.0556
	Unseen	0.0229	0.0292	0.0180	0.0435	0.0155	0.0257
EARec	All	0.1211	0.0976	0.1636	0.1221	0.0867	0.1052
	Unseen	0.0797	0.0535	0.1043	0.0677	0.0345	0.0331
NoContent	All	0.1493	0.1339	0.1823	0.1438	0.1057	0.1245
	Unseen	0.1361	0.0993	0.1680	0.0578	0.0648	0.0490

Variation sources. Table 6 separates model-run variation from mask variation. The mask SD is at most 0.0032 AUC at every rate and is zero at 0% and 100%, where all realizations coincide; differences between training runs are the dominant source of variation.

TREAD attention under replacement. At 100% replacement, the share of attention that each TREAD query assigns to the structured-feature token rises by 0.016 (CTR), 0.009 (CTCVR), and 0.074 (Play5) over the 0% values of 0.134, 0.159, and 0.337, and attention to the content tokens decreases accordingly. The Play5 query, which attends most to the structured-feature token, also shows the largest shift, in line with its larger AUC drop (Section 6).

B Pruning Trade-off

Figure 6 plots the pooled AUC and relative self-attention cost of Table 1 against the number of retained content tokens.

Table 6: Product-UNK pooled AUC with variation sources separated. Each cell is mean / model-run SD / mask-realization SD; model-run SD is computed over per-run means across masks, and mask SD over per-mask means across runs.

Model	Rate	CTR	CTCVR	Play5
TREAD	0%	0.6672 / 0.0005 / 0.0000	0.7174 / 0.0021 / 0.0000	0.6505 / 0.0019 / 0.0000
TREAD	25%	0.6642 / 0.0003 / 0.0003	0.7084 / 0.0026 / 0.0010	0.6285 / 0.0029 / 0.0010
TREAD	50%	0.6616 / 0.0003 / 0.0004	0.7011 / 0.0044 / 0.0009	0.6070 / 0.0105 / 0.0011
TREAD	75%	0.6597 / 0.0008 / 0.0001	0.6961 / 0.0065 / 0.0005	0.5877 / 0.0211 / 0.0006
TREAD	100%	0.6588 / 0.0028 / 0.0000	0.6974 / 0.0057 / 0.0000	0.5794 / 0.0345 / 0.0000
Single-vector	0%	0.6640 / 0.0016 / 0.0000	0.7149 / 0.0002 / 0.0000	0.6494 / 0.0024 / 0.0000
Single-vector	25%	0.6533 / 0.0010 / 0.0006	0.6957 / 0.0013 / 0.0007	0.6344 / 0.0034 / 0.0004
Single-vector	50%	0.6432 / 0.0019 / 0.0004	0.6779 / 0.0021 / 0.0007	0.6193 / 0.0051 / 0.0009
Single-vector	75%	0.6362 / 0.0021 / 0.0006	0.6627 / 0.0057 / 0.0007	0.6053 / 0.0059 / 0.0011
Single-vector	100%	0.6322 / 0.0031 / 0.0000	0.6537 / 0.0113 / 0.0000	0.5960 / 0.0071 / 0.0000
EARec	0%	0.6615 / 0.0015 / 0.0000	0.7115 / 0.0061 / 0.0000	0.6430 / 0.0031 / 0.0000
EARec	25%	0.6333 / 0.0015 / 0.0013	0.6793 / 0.0074 / 0.0008	0.6096 / 0.0050 / 0.0005
EARec	50%	0.6066 / 0.0043 / 0.0008	0.6474 / 0.0048 / 0.0010	0.5801 / 0.0062 / 0.0009
EARec	75%	0.5847 / 0.0051 / 0.0014	0.6179 / 0.0043 / 0.0009	0.5550 / 0.0064 / 0.0019
EARec	100%	0.5675 / 0.0100 / 0.0000	0.5955 / 0.0054 / 0.0000	0.5404 / 0.0052 / 0.0000
NoContent	0%	0.6584 / 0.0027 / 0.0000	0.7104 / 0.0009 / 0.0000	0.6350 / 0.0042 / 0.0000
NoContent	25%	0.6168 / 0.0028 / 0.0011	0.6667 / 0.0018 / 0.0025	0.5919 / 0.0028 / 0.0006
NoContent	50%	0.5827 / 0.0067 / 0.0021	0.6291 / 0.0041 / 0.0020	0.5579 / 0.0038 / 0.0008
NoContent	75%	0.5565 / 0.0105 / 0.0023	0.5973 / 0.0079 / 0.0032	0.5326 / 0.0055 / 0.0011
NoContent	100%	0.5414 / 0.0106 / 0.0000	0.5801 / 0.0096 / 0.0000	0.5141 / 0.0067 / 0.0000

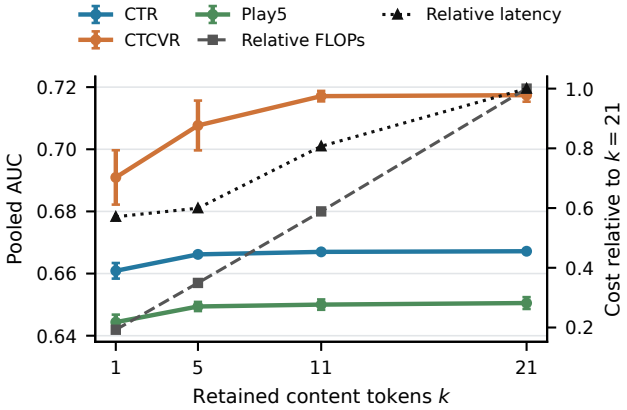


Figure 6: Inference-only TREAD pruning trade-off on the evaluation set. Solid lines show pooled AUC (left axis) with sample SD over three training runs; dashed and dotted lines show self-attention FLOPs and mean self-attention latency relative to the full reader (right axis), with latency from a fixed batch benchmark.